

Serving Map Tiles from a Raspberry Pi

A Portable Tile Server Workshop

FOSS4G Hiroshima 2026 / 2026-08-30 14:00–17:00 / Room 610

連絡系統 / Communication

- 質問は**挙手**でどうぞ。手を動かす時間・休憩中は直接声をかけてください

Questions: raise your hand, or grab me during hands-on time and breaks

- 進行中に調べたもの・話題に出た URL は、その場でスクリーンに表示し、終了後に**公開ページに追記**します

Links that come up will be shown on screen and added to the public page afterwards

- 資料はすべてここにあります / All materials live here:

alt9800.github.io/Raspi-Geo-Workshop

- 終了後の質問は GitHub リポジトリの **Issues** へ (日英どちらでも)

After today, questions go to GitHub Issues (Japanese or English)

翻訳と資料 / Translation and materials

- 講師は日本語で話します。Gemini による英訳をスクリーンに表示します
I will speak in Japanese; an English translation (Gemini) is shown on screen
- 翻訳ログは後からダウンロードできます
The translation log will be downloadable afterwards
- **今のうちに資料 PDF をダウンロードしてください**
Please download the PDF of these slides now
- 理由: 第三部でこの部屋はインターネットから切り離されます
Reason: in Part 3 this room goes offline — on purpose

自己紹介 / About me

(別資料で表示します / shown from a separate deck)

- 日本でフリーランスとして活動。リモートセンシングの企画、センシング機器・アプリ開発、VR コンテンツ、Fab スペース構築支援など
- 学生時代からの Raspberry Pi ユーザー。野外での観測・監視に多用してきました
- 受講料 7,500 円は講師報酬ではなく、すべてカンファレンス運営資金に充当されます
The 7,500 JPY fee goes entirely to conference operations, not to me

開講の経緯 / Why this workshop exists

- FOSS4G が広島に来るというタイミング。LOC の皆さんの尽力
- 先駆者がいます: 平澤さんらの **UNVT Portable**
 - Raspberry Pi でタイルを持ち運ぶという発想はすでに実践されてきました
- 私自身は学生時代から Raspberry Pi を野外に置いて使ってきました
 - ミツバチの巣箱の観測システム
 - 河川の水門の野外監視
- 「通信がない場所で地図を配る」を、今日この部屋で一通り体験してもらいます
Today you will build and hold the whole pipeline in your hands

動機 / Motivation: where the network ends

地図が一番必要な場所ほど、ネットワークがありません。

The places that need maps most are the places without connectivity.

- 山小屋 / mountain huts — 登山者に周辺の地形・ルートを配りたい
- 災害現場・紛争地 / disaster and conflict zones — 回線があてにならない
- 極域・遠隔地 / polar and remote regions — 衛星回線は高価で細い

そこで「箱ごと持っていくタイルサーバー」です。

So: a tile server you can carry in as a box.

PWA / ネイティブキャッシュとの比較

	PWA / アプリ内キャッシュ	ポータブルタイルサーバー
事前準備	端末ごとに必要	不要 (Wi-Fi に繋ぐだけ)
対象端末	準備した端末のみ	その場の全端末
データ更新	端末ごとに再配布	サーバー側のファイル 1 つ
容量の制約	端末ストレージ・OS の制限	SD カード次第 (数十 GB)
向く場面	個人の縦走・単独行動	拠点に人が集まる運用

端末に何も入れさせずに、その場の全員へ地図を配れる。これが今日のテーマです。

No app installs, no per-device caching — everyone connected gets the map.

本日のゴール / Today's goals

1. Raspberry Pi 上の **Martin** から **PMTiles** を配信し、ブラウザで地図を表示する
Serve PMTiles with Martin on a Raspberry Pi and view it in a browser
2. サーバーの状態を CLI で確認・操作できるようになる (systemd / curl / journalctl)
Inspect and control the server from the CLI
3. データを **ファイル 1 つの差し替え** で更新する
Update served data by swapping a single file
4. Pi を **Wi-Fi AP** 化し、スマホへ**完全オフライン**で地図を配る
Turn the Pi into a Wi-Fi AP and serve maps fully offline

予告: 最後に、この部屋のインターネットを本当に切ります。

Fair warning: at the end, we really do pull the plug.

全体像 / The big picture

[第一部] 座学：なぜ・何を・どうやって

|
[第二部] 手元の PC から Pi へ SSH

PC --(Opal ルーター)--> Raspberry Pi (Martin + PMTiles)

|
[第三部] Pi 自身が Wi-Fi AP になる

スマホ --(foss4g-pi-NN)--> Raspberry Pi ※インターネット非接続

- 第二部まで: Pi は「ネットワーク上のサーバー」
- 第三部から: Pi は「ネットワークそのもの + サーバー」

Raspberry Pi 基礎: 歴史 / A short history

- 2012 年、英国の Raspberry Pi 財団が教育用に発売。25–35 USD の「一枚板の PC」
Launched 2012 by the Raspberry Pi Foundation as an educational computer
- 教育用の想定を超えて、産業・ホビー・研究の定番に
Now a de-facto standard far beyond education: industry, hobby, research
- 系譜: Pi 1 → 2 → 3 → 4 → 5 と、小型系 (Zero / A+) や産業向け (Compute Module)
- 累計販売は数千万台規模。情報・作例・トラブル事例が世界中に蓄積されている
— 「困ったとき検索すれば誰かが同じ穴に落ちている」ことの価値は大きい

Raspberry Pi の種類と使い分け / The family

系統	例	特徴	向く用途
B 系	Pi 4B / 5	RAM 多め・有線 LAN・USB 多ポート	常設サーバー、デスクトップ
A 系	3A+	小型・省電力・ポート最小限	組み込み、今日の主役
Zero 系	Zero 2 W	最小・最安	センサーノード、ウェアラブル
CM 系	CM4 / CM5	基板に載せる産業向け	製品組み込み

同じ OS・同じ道具立てが全系統で動くので、開発は B 系で、現場投入は A 系や Zero で、という使い分けができます。

Raspberry Pi で何ができるか / What can it do?

私がこれまで野外で動かしてきた例:

- ミツバチ観測システム — 巣箱にカメラとセンサーを付けて挙動を記録
- 水門の野外監視 — 河川の水門を長期間、電源とネットワークが細い環境で監視

性能の感覚として:

- Pi 4 で ffmpeg を回しながら HLS 配信をしても消費電力は **約 3W**
Pi 4 transcoding with ffmpeg + HLS streaming: about 3 watts
- 3W はモバイルバッテリーやソーラーパネルで現実的に賄える数字です
3 W is realistic for battery or solar operation

タイル配信は動画処理よりずっと軽い。つまり今日の用途では余裕があります。

今日の主役: Raspberry Pi 3 Model A+

項目	仕様
SoC / RAM	BCM2837B0 (4 core, 1.4GHz) / 512MB
無線	CYW43455: 2.4GHz + 5GHz Wi-Fi, BT
ポート	USB-A x1, HDMI, microSD
有線 LAN	なし
電源	microUSB 5V/2.5A

制約こそが教材です:

- RAM 512MB → 重いスタックは載らない → **軽いサーバーを選ぶ理由**になる
- 有線 LAN なし → 無線設計を真面目にやる理由になる
- 5GHz 対応 → 第三部の AP 化で効いてきます

SD カードと電源 / SD cards and power

Raspberry Pi 運用の故障原因は、体感でほぼこの 2 つに集約されます。

SD カード

- OS もデータも全部 SD カード。品質差が露骨に出ます
- 選定目安: 信頼できるブランド / **A1 (Application Performance Class)** 以上
- 今日の構成: 16–32GB, A1。書き込みを減らすため swap 無効・ログは tmpfs

電源

- 電圧降下は不可解な不調の王様。ケーブルの質も効きます
- 今日の構成: 5V/2.5A 単ポートアダプタ + microUSB ケーブル

現物を回覧します。手に取ってみてください。

Hardware is circulating — please have a look.

OS の入れ方 / How the OS gets there

- Raspberry Pi は **OS のディスクイメージを microSD に書き込んで起動**します
The OS is a disk image written to a microSD card
- **Raspberry Pi Imager** (公式 GUI ツール) を使うと:
 - OS 選択 → SD 書き込みまで GUI で完結
 - 書き込み時に SSH 有効化・Wi-Fi 設定・ホスト名・ユーザーを事前設定できる
(= モニタもキーボードも繋がず、いきなり無線で入れる「headless」構成)
- CLI 派なら `dd` やイメージカスタマイズだけでも構成可能
- 今日の Pi: **Raspberry Pi OS Lite 64bit (Bookworm)** — GUI なし、CLI のみ

今日は自分で焼きます / You flash your own card

会場に焼きステーションを 2 箇所用意しています。

Two flashing stations are set up in the room.

- 講義と並行して順番に呼ぶので、呼ばれたら自分のカードを焼いてください
We'll call you up in turns during the lecture — flash your own card
- 手順 (ステーションに掲示もあります):
 - i. Raspberry Pi Imager で「Use custom」 → マスターイメージを選択
 - ii. 書き込み先 = ステーションのカードリーダーの SD
 - iii. カスタマイズ画面は **SKIP CUSTOMISATION** (設定は全部イメージ内)
 - iv. verify はそのまま有効で待つ (1 枚 15–20 分)
- 焼き上がるまでは講義を聞いていて OK。間に合わない席は隣と相席で進め、焼けた時点で自分の Pi に移ります

Pi への接続方法 / Ways to reach a Pi

- **SSH** — 今日のメイン。ターミナルからリモートのシェルに入る
- **VNC** — GUI デスクトップを飛ばす方式 (今日は Lite なので出番なし)
- 遠隔地の Pi に届く方法もいろいろあります:
 - ngrok / Cloudflare Tunnel — NAT の内側から外へトンネルを張る
 - **Tailscale** — WireGuard ベースのメッシュ VPN。最近の定番
- 64bit イメージなら **VS Code の Remote SSH** も使えます
 - Pi 側に VS Code Server が自動インストールされ、
手元の VS Code から Pi 上のファイルを直接編集できます

今日は同じ部屋にいたので、素直にローカルネットワークで SSH します。

タイル配信とは / What is tile serving?

- 地図は「全世界を 1 枚の画像」では扱えないので、
ズームレベル z / 座標 x, y で分割したタイルを並べて表示します
- クライアント (MapLibre など) は見えている範囲のタイルだけを要求する
`GET /tiles/{z}/{x}/{y}` — この仕組みが「タイル配信」です
- タイルの中身は 2 系統:
 - ラスタータイル — 描画済みの画像 (PNG/JPEG/WebP)
 - ベクトルタイル — 地物の座標と属性 (MVT)。描画はクライアント側
- ベクトルタイルはスタイルを後から変えられ、属性が引ける。今日はこちらが主役

タイルサーバー比較 / Choosing a tile server

	言語/実装	特徴	512MB の Pi では
GeoServer	Java	多機能の老舗。WMS/WFS 何でも	重い
TileServer GL	Node.js	スタイル込み配信、ラスタ化も	やや重い
pg_tileserv / Tegola	Go 等	PostGIS 前提	DB が要る
Martin	Rust	軽量・高速・単バイナリ。PMTiles 対応	適任

Martin を選ぶ理由:

- 単一バイナリを置くだけ。ランタイムも DB も不要
- メモリフットプリントが小さい (512MB でも余裕)
- **PMTiles** をそのまま配信できる

PMTiles とは / What is PMTiles?

- 数百万個のタイルを **1 ファイル** に格納するアーカイブ形式 (Protomaps 発)
A single-file archive holding millions of tiles
- 内部インデックスにより、HTTP **Range** リクエストで必要なタイルだけ読める
→ 静的ファイルとして置くだけでも配信が成立する (S3 でもローカルでも)
- 今日の運用に効く性質:
 - **配布が 1 ファイル** — コピーひとつでデータ更新が終わる
 - **追記も DB も不要** — SD カードに優しい (読み取り中心)
 - オフラインに強い — ネットが無くてもファイルがあれば地図になる
- 似た形式の MBTiles は中身が SQLite。PMTiles は単純な連続バイト列で、Range だけで読めるのが決定的な違いです

今日配信するデータ / Today's datasets

ファイル	内容	サイズ
hiroshima-base.pmtiles	ベースマップ (広島市+宮島, z0-14)	15MB
shelters.pmtiles	指定緊急避難場所	~1MB
hazard-flood.pmtiles	洪水浸水想定区域 (A31)	5MB
hazard-sediment.pmtiles	土砂災害警戒区域 (A33)	14MB
terrain.pmtiles	Terrain RGB (5m DEM)	38MB

- ベースマップは **planetiler** でローカル生成 (OSM データから約 6 分)
Basemap generated locally with planetiler (~6 min from OSM data)
- 生成・変換はデータリポジトリに同梱の `scripts/01` ~ `04` で全て再現できます
- 各タイルの中身・出典・ライセンスの一覧: **catalog.html**
生成パイプラインの解説: **data-pipeline.md**

データの出典 / Data sources and attribution

- © OpenMapTiles © OpenStreetMap contributors
- 「指定緊急避難場所データ」(国土地理院) をもとに作成
Derived from "Designated Emergency Evacuation Site Data", GSI Japan
- 「国土数値情報 (洪水浸水想定区域データ)」(国土交通省) をもとに作成
Derived from "National Land Numerical Information (Flood Inundation Zones)", MLIT Japan
- 「国土数値情報 (土砂災害警戒区域データ)」(国土交通省) をもとに作成
Derived from "National Land Numerical Information (Landslide Warning Zones)", MLIT Japan
- 「基盤地図情報 数値標高モデル」(国土地理院) をもとに作成
Derived from "Fundamental Geospatial Data (DEM)", GSI Japan

オフライン配信でも出典表示は消さない — 同梱ビューフは配信中のソースに応じて自動表示します。

休憩前の宿題 (1/2): Pi を起動する / Power up your Pi

焼き上がったカードを席の Raspberry Pi 3A+ に挿して起動します。

(まだ焼けていない席は、焼けた時点で同じ手順を踏めば追いつけます)

1. 席の Pi と電源アダプタを確認 (席番号ラベル付き)。SD カードを挿す
2. microUSB を挿して**電源投入** — 電源スイッチはありません。挿したら起動です
3. LED の見方:
 - 赤 (PWR): 通電中。点きっぱなしで正常
 - 緑 (ACT): SD カードアクセス。起動中はチカチカします
4. 1〜2 分でネットワークに参加し、SSH 待ち受け状態になります

モニタもキーボードも繋ぎません。ここから先は全部ネットワーク越しです。

休憩中に講師が全台の起動を確認します。

休憩前の宿題 (2/2): SSH クライアント / Before the break

SSH クライアントをいま起動しておいてください

Please get your SSH client running now.

- macOS / Linux: ターミナルがあれば OK (`ssh` は最初から入っています)
- Windows: PowerShell / Windows Terminal の `ssh`、または PuTTY
- 動作確認はこれだけ:

```
ssh -V
```

バージョンが出れば準備完了です。困ったら休憩中に声をかけてください。

休憩 / Break (14:50–15:00)

再開後、実際に Pi へ接続します。

After the break, we connect to the Pis.

まずは手元だけで / PMTiles, locally first

サーバーの前に「PMTiles は 1 ファイルで地図になる」を体感します。

1. 配布した `shelters.pmtiles` を用意
(公開ページ alt9800.github.io/Raspi-Geo-Workshop/shelters.pmtiles から DL)
2. ブラウザで **pmtiles.io** を開く
3. ファイルをドラッグ & ドロップ

→ サーバー無しで避難場所が地図に描かれます。

ビューワがファイル内のインデックスを直接読んでいるだけです。

これをネットワーク越しに配れるようにするのが、ここからの作業です。

Now let's do the same thing over the network.

席ラベルと SSH 接続 / Seat labels and SSH

- 各席のラベル: 席番号 NN = IP アドレスの末尾
Seat number NN = last part of the IP address
- IP は `10.x.x.1NN` の規約です (席 03 なら末尾 103)

```
ssh workshop@10.x.x.1NN
```

- 初回は `known_hosts` の確認が出ます → `yes`
- パスワードは席のラベルに記載
- 接続できたら:

```
hostname          # クローン直後は全台 pi-00 と出ます (正常)
ip a               # 自分の IP を確認 - こちらは席番号どおり
```

- IP が席番号どおりになるのは、ルーターが **Pi 本体の MAC アドレス** に IP を固定しているからです。SD カードの中身には依存しません
- `hostname` はこの後の「個体化」で直します。
うまくいかない場合は救済タイムで対応します。

個体化 / Make it yours

全員のカードは同じイメージのクローンなので、名前を自分の席番号にします。

Every card is a clone — time to give yours its own identity.

```
sudo /opt/scripts/personalize-sd.sh NN    # NN = あなたの席番号 (01-15)
sudo reboot
```

- 1～2 分待って SSH し直し、`hostname` が `pi-NN` になったのを確認
- このスクリプトがやっていること:
 - `hostname` を `pi-NN` に変更
 - 第三部で使う AP の SSID (`foss4g-pi-NN`) とチャンネルを席番号で設定
- これをやらないと第三部で全員の AP が同名・同チャンネルで衝突します。
必ずここで済ませてください

サービスとして動いている / It runs as a service

```
systemctl status martin
```

- `active (running)` — Martin は **systemd** のサービスとして動いています
- 「サービスとして動く」の意味:
 - 起動時に自動で立ち上がる (誰もログインしなくても)
 - 落ちたら自動で再起動する (`Restart=on-failure`)
 - ログが `journald` に集約される
- 手で `./martin` を叩く運用との違いが、第三部の「無人復帰」の土台になります
This is what makes unattended recovery possible in Part 3

HTTP で聞いてみる / Talking to Martin over HTTP

```
curl http://localhost:3000/health  
curl http://localhost:3000/catalog | jq
```

- `/health` — 生死確認。監視の基本
- `/catalog` — 配信中のタイルソース一覧が JSON で返る
- `jq` は JSON の整形・抽出ツール。読み方:
 - キー: `"tiles"` の下に各ソース
 - `content_type` や `description` で中身の見当がつく

「ブラウザで見る前に curl で確かめる」は、サーバー運用の基本動作です。

中身を覗く / Under the hood

```
ls -lh /opt/tiles/  
cat /opt/tiles/config.yaml
```

- PMTiles の実体と Martin の設定がここにあります
- ログを流しっぱなしにしてみます:

```
journalctl -u martin -f
```

- この状態でブラウザから地図を開くと、**自分のアクセスがログに流れるのが見えます**
- `-u martin` でサービスを絞り、`-f` で追尾 (follow)

ブラウザで地図を表示 / The map, served by your Pi

ブラウザで開く:

```
http://10.x.x.1NN/
```

- さっき pmtiles.io で見たのと同じデータが、今度はあなたの Pi から来ています
- DevTools の Network タブで確認:
 - タイルのリクエスト先が `10.x.x.1NN` になっている
 - 1 タイルずつ取得されている様子が見える
- ビューワは MapLibre GL JS の単一 HTML。これも Pi が配信しています

属性を読む / Click to inspect

- 地図上の避難場所をクリック → 属性がポップアップします
- ベクトルタイルは「絵」ではなく「データ」なので、地物ごとの属性が引けます
Vector tiles carry attributes, not pixels
- 見てほしい属性:
 - 名称 / name — 施設名
 - 対応災害の区分 — どの災害のときに使える避難場所か
- 「洪水のときに使える場所か」を、この後の演習で地図として重ねます

差し替え演習 / The swap exercise

データ更新が「ファイル 1 つ」で済むことを体験します。

```
sudo mv /opt/tiles/hazard.pmtiles.bak /opt/tiles/hazard.pmtiles
sudo systemctl restart martin
curl http://localhost:3000/catalog | jq
```

- ブラウザを再読み込み → **洪水浸水想定レイヤ**が出現します
- 見てほしいこと: 避難場所と浸水想定区域の**重なり**
 - 浸水域の中にある避難場所はどれか
 - 「洪水対応」の属性と整合しているか
- 現場でのデータ更新もこれと同じです: ファイルを置いて restart するだけ
- 洪水版と土砂版の違いはデータリポジトリの docs/hazard-comparison.md 参照

追いつきタイム / Catch-up time

ここまでで詰まっている方を優先します。挙手でどうぞ。

先に進みたい方への小課題:

- DevTools の Network タブでタイルを 1 つ選び、確認してみてください:
 - **Content-Type** は何になっているか
 - 1 タイルの**サイズ**はどのくらいか (数 KB~数十 KB)
 - ズームレベルによってサイズはどう変わるか
- `curl -sI` で同じことを CLI から確認できます:

```
curl -sI http://localhost:3000/hazard/10/900/400 | head
```

閑話: DEM と遺跡発見ブーム / Aside: DEM archaeology

- 日本では DEM (数値標高モデル) の公開が進んだ結果、
空前の遺跡発見ブームが起きています
- 5m 精度の DEM を陰影図にすると、地表の微妙な起伏が浮かび上がる:
 - 古い城の城壁・曲輪の跡
 - 近世より前の墓や土塁
- 森に覆われていても地形は残る — 航空レーザ測量は樹冠を「透かして」地面を測れます
- 文化財の研究者も今回の FOSS4G にかなり参加しています。
懇親会などで聞いてみると面白いはずです
- 今日の terrain.pmtiles も同じ 5m DEM 由来です

休憩 / Break (15:50–16:00)

次が本日の山場です。

The main event is next.

ここからネットを使いません / Going offline

- 第三部では、この部屋はインターネットを使いません
- 資料はダウンロード済みの PDF、または **Pi の中の clone** を参照してください
(Pi 自身が資料一式を配信しています)

AP 化の図解:

いままで:	PC → Opal ルーター → Pi	(Pi はクライアント)
これから:	スマホ → Pi (それ自体が AP)	(Pi がネットワークの親)

- **片道です:** 一度 AP になった Pi は、再起動しても AP として上がります
- 切り替えた瞬間に **SSH が切れますが、それは正常です**
Your SSH session will drop. That is expected, not a failure.

AP 化を実行 / Flip the switch

全員で実行します:

```
sudo /opt/scripts/switch-to-ap.sh
```

- SSH が切れます (前のスライドの通り、正常)
- 1〜2 分待つと、Wi-Fi 一覧に SSID **foss4g-pi-NN** が現れます (NN はあなたの席番号)
- スクリプトがやっていること:
 - クライアント接続 (Opal 向け) の自動接続を無効化
 - AP プロファイルを自動接続 **有効**で起動
 - 以後、再起動すれば無条件で AP として上がる

スマホで受信 / Receive it on your phone

1. 席の **QR コード** をスマホで読む → あなたの Pi の AP に接続
2. ブラウザで開く:

```
http://192.168.4.1/
```

3. さっきと同じ地図が表示されます — ただし今度は、
経路上にインターネットが一切ありません
The same map — with no internet anywhere in the path
- `192.168.4.1` は AP モードの Pi 自身のアドレスです
 - 何台でも同時に繋がります。周りの人と一緒にどうぞ

救済タイム / Rescue time

- AP が立ち上がらない個体は、隣の方と相席してください
If your AP did not come up, please pair with a neighbor
- できた方への追加課題:
 - **隣の席の AP** (foss4g-pi-MM) にも繋いでみてください
 - 同じビューワ・違う Pi から地図が来ることを確認
- 電波の混雑を避けるため、各 Pi は 5GHz の別チャンネルに分散しています
(種明かしは後ほど)

本日の山場 / The main event

これから、この部屋の上流ルーター (Opal) の電源を切ります。

- 上流が消えます。インターネットへの経路は完全になくなります
- それでも —

地図は動き続けます。

The map keeps working.

- あなたのスマホと Pi の間で完結しているからです
- 「持ち運べるタイルサーバー」が成立した瞬間です

電源引き抜き演習 / The power-pull exercise

サーバーの電源をいきなり抜きます。現場では日常です。

1. Pi の **microUSB** を引き抜く (シャットダウン操作なし)
2. 10 秒待って差し直す
3. 1～2 分待つ → SSID `foss4g-pi-NN` が勝手に復活
4. スマホを繋ぎ直す → 地図も復活
 - 誰も操作していないのに戻ってくる = 無人復帰
 - 乱暴に見えますが、これに耐える構成にしてあるのがポイントです
(読み取り中心の PMTiles + 書き込みを減らした OS 設定)

なぜ勝手に復帰するのか / Why it comes back

復帰を支えている 2 つの仕組み:

systemd

- Martin はサービスとして登録済み → 起動シーケンスで自動起動
- 落ちても `Restart=on-failure` で再起動

NetworkManager の自動接続プロファイル

- AP プロファイルが `autoconnect` 有効 → 起動すれば AP が立つ
- 「片道スイッチ」の実体は、この `autoconnect` フラグの付け替えです

電源投入 → OS 起動 → AP 起動 → Martin 起動 → 配信再開。

この間、人間は何もしていません。

全員の SSID 復活を確認します。まだの方は挙手を。

現地運用の設計論 / Designing for the field

今日の構成を現場に持っていくときの論点:

- **接続 UX** — QR コードで SSID 接続まで誘導。captive portal 風の
リダイレクトで「繋いだら地図が開く」まで作り込む手もある
- **死活監視** — `/health` を定期的に叩く。今日私が休憩中に回していた
`check-all.sh` はまさにこれ (全台の health を走査)
- **電源** — 3W 級なのでモバイルバッテリーで半日は現実的。ソーラーも視野
- **SD カード対策** — swap 無効・ログ tmpfs で書き込みを減らす (今日の構成)
- **電波設計の種明かし** — 本日は 5GHz W52 帯で、席グループごとに
ch 36/40/44/48 へ分散していました。2.4GHz 15 台密集は破綻します

閑話: 調達がボトルネックだった話 / Aside: the supply crunch

このワークショップ準備で最大のボトルネックは、技術ではなく**調達**でした。

- データセンター需要による世界的な品薄が、末端の機材にまで波及しています
- MacBook の RAM すら在庫が細く、MacBook Air は 64GB 構成が選べない状態。
Mac mini も同様
- 余波はマイコンにも: **Raspberry Pi Zero 2 W** は日本では **11 月まで入手困難**
という状況でした
- 「現場で使う機材は、使う予定の semester より 1 つ前に確保する」が
実運用の教訓です

Extra: 持ち帰り課題 / Take-home extras

extra.md に手順、データリポジトリの `scripts/` と `data-pipeline.md` に実体があります。
すべて自宅で再現可能:

- **タイル生成 (PC 側)** — planetiler で OSM からベースマップを自作 (約 6 分)
- **DEM → Terrain RGB** — 基盤地図情報の DEM から地形タイルを生成
- **Maputnik** — ブラウザでスタイルを編集して、見た目を自分好みに
- **autohotspot** — クライアント/AP を自動で行き来する構成 (今日は片道でした)
- **土砂版差し替え** — `hazard-sediment.pmtiles.bak` で差し替え演習をもう一周
(mv で `.bak` を外して restart、洪水版と同じ手順)

質疑 / Q&A

- 挙手でどうぞ。ハンズオン中に受けて保留にした質問もここで拾います
- この後のクロージングでもう一度、連絡先とリポジトリを提示します

まとめ / Wrap-up: four takeaways

1. 箱で持ち運べる — サーバーも地図も、片手に載る箱に収まる
The whole stack fits in one hand
2. 端末に準備が要らない — 繋いだ端末すべてに配れる
Client devices need zero preparation
3. 更新はファイル 1 つ — mv と restart で終わる
Updating means swapping one file
4. 無人で復帰する — 電源を抜いても勝手に戻る
It recovers with nobody watching

おわりに / Closing

- Pi の回収 — 席番号と照合しながら回収します。ご協力ください
- アンケート — QR を表示します。運営へのフィードバックになります
- 後日の質問は **GitHub Issues** へ — 日英どちらでも歓迎です
Questions after today: GitHub Issues, Japanese or English welcome
- リポジトリ — スライド・スクリプト・データ生成手順すべてここに:

github.com/alt9800/Raspi-Geo-Workshop

ありがとうございました。

Thank you — and may your maps work where the network doesn't.